

RAG-Based Customer Support Assistant using LangChain, LangGraph & HITL

1. Introduction

The objective of this project is to design and build an **AI-powered Customer Support Assistant** using the **Retrieval-Augmented Generation (RAG)** architecture.

The system takes user queries and retrieves relevant information from a knowledge base (PDF) to generate accurate and context-aware responses.

This project also integrates:

- **LangChain** for pipeline building
- **LangGraph** for workflow orchestration
- **Human-in-the-Loop (HITL)** for escalation

2. Problem Statement

Traditional chatbots often:

- Give incorrect answers
- Hallucinate responses
- Lack real-time knowledge

To solve this, we build a system that:

Retrieves real information → Uses LLM → Generates accurate answers

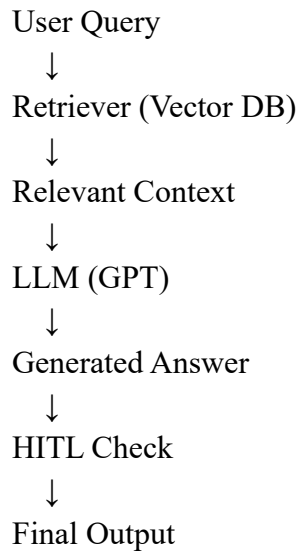
3. System Architecture (HLD – High Level Design)

Components

1. User Interface (CLI / Input)
2. PDF Knowledge Base
3. Document Loader
4. Text Splitter (Chunking)
5. Embedding Model
6. Vector Database (ChromaDB)

7. Retriever
8. LLM (OpenAI)
9. LangGraph Workflow
10. HITL Module

Architecture Flow



4. RAG Pipeline Explanation

◆ Step 1: Document Loading

The system loads a PDF file using a document loader.

Purpose:

Convert raw PDF into readable text documents

◆ Step 2: Chunking

Large documents are split into smaller chunks.

Chunk Size: 500

Overlap: 50

This ensures:

- Better retrieval accuracy

- Efficient embeddings

◆ Step 3: Embeddings

Text chunks are converted into vectors using embedding models.

Model: OpenAI Embeddings

Purpose:

- Convert text → numerical representation

◆ Step 4: Vector Database

Chunks are stored in **ChromaDB**.

Purpose:

- Fast similarity search
- Efficient retrieval

◆ Step 5: Retrieval

When a query is asked:

- Relevant chunks are retrieved
- Based on similarity

◆ Step 6: LLM Response Generation

The LLM generates answers using:

Context + User Query → Answer

Rules:

- Answer ONLY from context
- Avoid hallucination

5. LangGraph Workflow

LangGraph is used to define the system flow.

Nodes

1. Process Node

- Performs RAG retrieval + LLM response

2. Decision Node

- Checks if response is valid

Flow

Process Node → Decision Node → Output

6. Human-in-the-Loop (HITL)

Purpose

To handle cases where:

- Answer is missing
- Answer is weak
- Context is insufficient

Logic

IF:

- "Not found" in answer
- OR answer length < threshold

THEN:

→ Escalate to human

Output Example

Escalated to human agent

7. Low-Level Design (LLD)

Modules

1. Loader Module

- Loads PDF data

2. Chunking Module

- Splits text

3. Embedding Module

- Converts text to vectors

4. Retriever Module

- Fetches relevant chunks

5. RAG Pipeline Module

- Combines retrieval + LLM

6. LangGraph Module

- Controls flow

7. HITL Module

- Handles escalation

Data Structure

```
state = {  
    "query": str,  
    "answer": str  
}
```

8. Technologies Used

- Python
- LangChain
- LangGraph
- OpenAI API
- ChromaDB
- PyPDF

9. Challenges Faced

- Handling long documents
- Avoiding hallucination

- Designing proper prompts
- Managing API usage
- Ensuring relevant retrieval

10. Advantages of the System

- Context-aware responses
- Reduced hallucination
- Scalable architecture
- Modular design
- Explainable outputs

11. Limitations

- Depends on PDF quality
- Requires API access
- Latency due to multiple steps
- Retrieval accuracy varies

12. Future Improvements

- Use advanced vector DB (FAISS, Pinecone)
- Add UI (Streamlit / Web App)
- Improve ranking algorithm
- Multi-document support
- Multi-agent systems

13. Conclusion

This project demonstrates how to build a **production-level AI system** using RAG architecture.

Key learnings:

- Combining retrieval with LLM improves accuracy

- LangGraph enables structured workflows
- HITL ensures reliability

This system can be extended to real-world applications like:

- Customer support
- Knowledge assistants
- Enterprise AI systems